

Introduction to APIs

Building Blocks of Modern Software



What is an API?

API = Application Programming Interface

Think of it as a waiter in a restaurant:

- • You (the client) don't go into the kitchen
- • The waiter (API) takes your order to the kitchen (server)
- • The kitchen prepares your meal (processes request)
- • The waiter brings back your food (returns response)

Why Use APIs?



Integration

Connect different software systems seamlessly



Data Access

Retrieve data without knowing database structure



Modularity

Reuse code across multiple applications



Scalability

Build once, use everywhere

Types of APIs

REST APIs

- • Most common type
- • Uses HTTP methods (GET, POST, PUT, DELETE)
- • Returns JSON or XML
- • Stateless

Example: OpenWeatherMap, Twitter API

Other Types

- • SOAP APIs - XML-based, enterprise
- • GraphQL - Query language for APIs
- • WebSocket - Real-time bidirectional
- • gRPC - High-performance RPC

HTTP Methods: The API Verbs

Method	Purpose	Example
GET	Retrieve data	<i>Get user information</i>
POST	Create new data	<i>Create a new user</i>
PUT	Update existing data	<i>Update user profile</i>
DELETE	Remove data	<i>Delete a user account</i>

Anatomy of an API Request

1. Base URL:

`https://api.example.com`

2. Endpoint:

`/users/123`

3. Parameters:

`?format=json&limit=10`

4. Headers:

`Authorization: Bearer YOUR_API_KEY`

API Authentication



Type	Description	Security
API Keys	Simple token in header or URL	Basic
OAuth 2.0	Industry standard, token-based	High
JWT	JSON Web Tokens, stateless	High
Basic Auth	Username/password encoded	Low

HTTP Status Codes

Common responses you'll encounter:

2xx Success

200 OK - Request succeeded • 201 Created - New resource created • 204 No Content - Success, no data returned

4xx Client Errors

400 Bad Request - Invalid syntax • 401 Unauthorized - Authentication required • 404 Not Found - Resource doesn't exist

5xx Server Errors

500 Internal Server Error - Server malfunction • 503 Service Unavailable - Server overloaded

JSON: The Language of APIs

```
{  
  "user_id": 12345,  
  "name": "Jane Doe",  
  "email": "jane@columbia.edu",  
  "roles": ["student", "researcher"],  
  "active": true,  
  "metadata": {  
    "department": "QMSS",  
    "year": 2026  
  }  
}
```

Making API Calls in Python

Using the requests library:

```
import requests

# Making a GET request
url = "https://api.example.com/users/123"
headers = {"Authorization": "Bearer YOUR_KEY"}

response = requests.get(url, headers=headers)

# Parse the response
data = response.json()
print(data["name"])
```

Error Handling Best Practices

```
try:
    response = requests.get(url, timeout=10)
    response.raise_for_status() # Raises HTTPError for bad status
    data = response.json()

except requests.exceptions.Timeout:
    print("Request timed out")

except requests.exceptions.HTTPError as e:
    print(f"HTTP Error: {e}")

except requests.exceptions.RequestException as e:
    print(f"Error: {e}")
```

Rate Limiting and Best Practices

Why Rate Limits Exist

- Prevent server overload
- Ensure fair usage
- Protect against abuse

Common Limits:

- 100 requests/hour
- 1000 requests/day

```
# Respect rate limits
import time

for item in items:
    response = api_call()
    time.sleep(1) # Wait 1s

# Check headers
limit = response.headers[
    "X-RateLimit-Remaining"
]
```

Working with Paginated Data

Many APIs return large datasets in pages:

```
page = 1
all_results = []

while True:
    url = f"https://api.example.com/data?page={page}"
    response = requests.get(url)
    data = response.json()

    all_results.extend(data["results"])

    if not data["next"]: # No more pages
        break
    page += 1
```

API Wrappers: Making Life Easier

Instead of raw HTTP requests, use wrapper libraries:

Without Wrapper

```
url = base + endpoint
headers = {...}
params = {...}
response = requests.get(
    url, headers, params
)
data = response.json()
```

With Wrapper

```
# Much simpler!

client = APIClient(
    api_key=YOUR_KEY
)

data = client.get_data()
```

Reading API Documentation

Key sections to look for:

1

Authentication

How to get and use API keys

2

Endpoints

Available URLs and what they do

3

Parameters

Required and optional inputs

4

Response Format

Structure of returned data

5

Rate Limits

Request quotas and restrictions

6

Error Codes

What went wrong and why

Common API Mistakes to Avoid

- ✓ Hard-coding API keys in code (use environment variables)
- ✓ Not handling errors or timeouts
- ✓ Ignoring rate limits (can get your access blocked)
- ✓ Not validating/sanitizing user input in requests
- ✓ Forgetting to close connections or clean up resources
- ✓ Not checking API version for breaking changes
- ✓ Storing sensitive data from responses insecurely

API Best Practices

For API Consumers

- • Use environment variables for keys
- • Implement retry logic with backoff
- • Cache responses when appropriate
- • Log requests for debugging
- • Use wrapper libraries when available
- • Always validate response data

For API Developers

- • Provide clear, comprehensive docs
- • Use semantic versioning
- • Return meaningful error messages
- • Implement proper authentication
- • Set reasonable rate limits
- • Follow REST conventions

Useful Tools and Resources

Development Tools

- Postman - API testing platform
- Insomnia - REST client
- Swagger/OpenAPI - API specs
- cURL - Command line HTTP client
- httpie - User-friendly HTTP CLI

Learning Resources

- Python requests docs
- RESTful API design guidelines
- Public APIs list ([github.com/public-apis](https://github.com/public-apis/public-apis))
- API design patterns
- OAuth 2.0 specification

Time for Hands-On Practice!

We'll work through examples in the Jupyter notebook:

- 1 Making your first API call with NewsAPI
- 2 Parsing JSON responses and extracting data
- 3 Handling different HTTP status codes
- 4 Error handling and retry logic
- 5 Building a simple wrapper function

Key Takeaways

- ✓ APIs enable software communication and integration
- ✓ REST is the most common API architecture
- ✓ Authentication and rate limiting are crucial
- ✓ Good error handling makes robust applications
- ✓ Practice with real APIs to build confidence

Thank You!

Questions?